

PCE Local Deployment Guide

Run Your Price Comparison Engine on a Local Server for the First 2 Months

1. Hardware Requirements

Component	Minimum	Recommended	Notes
RAM	16 GB	32 GB	Playwright + PostgreSQL + pgVector is hungry
CPU	4 cores (Intel i5 / AMD Ryzen 5)	6-8 cores	More cores = more parallel scrapes
Storage	256 GB SSD	512 GB NVMe SSD	PostgreSQL with 90-day history grows fast
OS	Ubuntu 22.04 LTS	Ubuntu 24.04 LTS	Docker runs best on Linux
Internet	50 Mbps	100 Mbps+	Upload speed matters for Cloudflare Tunnel
UPS	600 VA	1,100 VA	Prevents data corruption during power cuts

My setup suggestion: A used Dell OptiPlex 7050 (i5-7500, 32GB RAM, 512GB NVMe) costs ~ 18,000 on OLX and handles this stack comfortably.

2. Install Docker & Docker Compose

```
# Update system
sudo apt update && sudo apt upgrade -y

# Install Docker
sudo apt install -y ca-certificates curl gnupg lsb-release
sudo mkdir -p /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o /etc/apt/keyrings/
  ⇐ docker.gpg

echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] https://
  ⇐ download.docker.com/linux/ubuntu $(lsb_release -cs) stable" | sudo tee /etc/apt/sources.list.d/
  ⇐ docker.list > /dev/null

sudo apt update
```

```
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-compose-plugin

# Add your user to docker group (logout/login required after)
sudo usermod -aG docker $USER
newgrp docker

# Verify
docker --version
docker compose version
```

3. Cloudflare Tunnel Setup (The Magic Sauce)

This exposes your local server to the internet with HTTPS — no port forwarding, no static IP.

Step 3A: Create a Cloudflare Account

1. Go to dash.cloudflare.com
2. Add your domain (e.g., `pce-api.io`) — you can buy one for ~ 800/year on Namecheap/Cloudflare
3. Change your domain's nameservers to Cloudflare's (they'll tell you which ones)

Step 3B: Install cloudflared

```
# Download and install cloudflared
wget -q https://github.com/cloudflare/cloudflared/releases/latest/download/cloudflared-linux-amd64.deb
sudo dpkg -i cloudflared-linux-amd64.deb

# Verify
cloudflared --version
```

Step 3C: Authenticate & Create Tunnel

```
# Login — this opens a browser for OAuth
cloudflared tunnel login

# Create a tunnel named "pce-local"
cloudflared tunnel create pce-local

# Note the tunnel ID printed — save it! It looks like:
# Tunnel credentials written to /home/user/.cloudflared/<UUID>.json
# Keep that UUID safe — you'll need it
```

Step 3D: Create Config File

```
mkdir -p ~/.cloudflared

# Replace <YOUR-TUNNEL-UUID> with the actual UUID from above
# Replace pce-api.io with your actual domain

cat > ~/.cloudflared/config.yml << 'EOF'
```

```

tunnel: <YOUR-TUNNEL-UUID>
credentials-file: /home/$USER/.cloudflared/<YOUR-TUNNEL-UUID>.json

ingress:
  # API Server
  - hostname: api.pce-api.io
    service: http://localhost:8000

  # Grafana Dashboard
  - hostname: grafana.pce-api.io
    service: http://localhost:3000

  # Flower (Celery monitoring)
  - hostname: flower.pce-api.io
    service: http://localhost:5555

  # Prometheus
  - hostname: prometheus.pce-api.io
    service: http://localhost:9090

  # Fallback
  - service: http_status:404
EOF

```

Step 3E: Add DNS Records

```

# Create DNS routes for each subdomain
cloudflared tunnel route dns pce-local api.pce-api.io
cloudflared tunnel route dns pce-local grafana.pce-api.io
cloudflared tunnel route dns pce-local flower.pce-api.io
cloudflared tunnel route dns pce-local prometheus.pce-api.io

```

Step 3F: Run as a Systemd Service (Auto-Start)

```

# Install the service
sudo cloudflared service install
sudo cp ~/.cloudflared/config.yml /etc/cloudflared/config.yml
sudo cp ~/.cloudflared/<YOUR-TUNNEL-UUID>.json /etc/cloudflared/

# Edit the service config to point to the right creds
sudo nano /etc/cloudflared/config.yml
# Make sure credentials-file path is absolute: /etc/cloudflared/<UUID>.json

# Start and enable
sudo systemctl start cloudflared
sudo systemctl enable cloudflared
sudo systemctl status cloudflared

# Check logs if issues
sudo journalctl -u cloudflared -f

```

Test it: Open <https://api.pce-api.io/health> in your browser. You should see the FastAPI health response.

4. Prepare Your PCE Repository

```
# Clone your repo
cd ~
git clone https://github.com/mukro/price_comparision_engine.git
cd price_comparision_engine

# Copy env template
cp .env.example .env

# Edit .env with your local settings
nano .env
```

Key .env Values for Local Deployment

```
# Database — internal Docker network, no need for public exposure
POSTGRES_USER=price_engine
POSTGRES_PASSWORD=<STRONG_PASSWORD_20_CHARS>
POSTGRES_HOST=db
POSTGRES_PORT=5432
POSTGRES_DB=price_comparison

# Redis — internal only
REDIS_URL=redis://:your_redis_password@redis:6379/0

# JWT — generate a strong secret
JWT_SECRET_KEY=<openssl rand -hex 32>
JWT_ALGORITHM=HS256
JWT_EXPIRE_MINUTES=60

# Admin credentials — CHANGE THESE
ADMIN_EMAIL=admin@pce-api.io
ADMIN_PASSWORD_HASH=$2b$12$... # bcrypt hash of your password

# CORS — allow your Flutter app and local dev
ALLOWED_ORIGINS=https://api.pce-api.io,http://localhost:3000

# Email — use SendGrid or keep empty for local dev
SENDGRID_API_KEY=your_sendgrid_key
FROM_EMAIL=alerts@pce-api.io

# Scraping — start conservative
SCRAPING_ENABLED=true
DEFAULT_SCRAPE_RPM=6
ENFORCE_ROBOTS_TXT=true
ENFORCE_DOMAIN_ALLOWLIST=false
```

```
# FCM — add when you configure Firebase
# FCM_SERVER_KEY=...
```

5. Modify docker-compose.yml for Local Production

Your existing docker-compose.yml is almost perfect. Just a few tweaks for local stability:

```
version: "3.8"

services:
  db:
    image: pgvector/pgvector:pg16
    container_name: price_db
    restart: unless-stopped # Changed: always → unless-stopped
    env_file: .env
    environment:
      POSTGRES_DB: ${POSTGRES_DB}
      POSTGRES_USER: ${POSTGRES_USER}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
    # REMOVE the ports exposure — only internal access via Docker network
    # ports:
    #   - "127.0.0.1:5432:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data
      - ./scripts/init.sql:/docker-entrypoint-initdb.d/init.sql
      - ./app/db/schema.sql:/docker-entrypoint-initdb.d/02_schema.sql
      - ./monetization_foundation_v1.sql:/docker-entrypoint-initdb.d/03_monetization.sql
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER} -d ${POSTGRES_DB}"]
      interval: 10s
      timeout: 5s
      retries: 5
    networks:
      - app_network
    # Add resource limits
    deploy:
      resources:
        limits:
          memory: 4G
        reservations:
          memory: 2G

  redis:
    image: redis:7-alpine
    container_name: price_redis
    restart: unless-stopped
    env_file: .env
    command: >
```

```

    sh -c "redis-server --requirepass ${REDIS_PASSWORD} --bind 0.0.0.0 --maxmemory 512mb
    ↪ --maxmemory-policy allkeys-lru"
# NO ports exposed — internal only
volumes:
  - redisdata:/data
healthcheck:
  test: ["CMD", "redis-cli", "-a", "${REDIS_PASSWORD}", "ping"]
  interval: 10s
  timeout: 5s
  retries: 5
networks:
  - app_network
deploy:
  resources:
    limits:
      memory: 1G

api:
  build:
    context: .
    dockerfile: Dockerfile
  container_name: price_api
  restart: unless-stopped
# REMOVE --reload for production
command: uvicorn app.main:app --host 0.0.0.0 --port 8000 --workers 2
# NO ports exposed — accessed via Cloudflare Tunnel
env_file: .env
environment:
  - PYTHONUNBUFFERED=1
volumes:
  - ./app
depends_on:
  db:
    condition: service_healthy
  redis:
    condition: service_healthy
healthcheck:
  test: ["CMD", "curl", "-f", "http://localhost:8000/health"]
  interval: 30s
  timeout: 10s
  retries: 3
  start_period: 15s
networks:
  - app_network
deploy:
  resources:
    limits:
      memory: 2G

celery_worker:
  build:

```

```

    context: .
    dockerfile: Dockerfile
container_name: price_celery_worker
restart: unless-stopped
command: >
    celery -A app.celery_app:celery_app worker
    -Q default
    --loglevel=info
    --concurrency=2
    -n worker_default@%h
env_file: .env
environment:
    - PYTHONUNBUFFERED=1
volumes:
    - ./app
    - model_cache:/root/.cache
depends_on:
    db:
        condition: service_healthy
    redis:
        condition: service_healthy
networks:
    - app_network
deploy:
    resources:
        limits:
            memory: 3G # Playwright needs RAM

celery_worker_priority:
    build:
        context: .
        dockerfile: Dockerfile
    container_name: price_celery_worker_priority
    restart: unless-stopped
    command: >
        celery -A app.celery_app:celery_app worker
        -Q high_priority
        --loglevel=info
        --concurrency=1
        -n worker_priority@%h
    env_file: .env
    environment:
        - PYTHONUNBUFFERED=1
    volumes:
        - ./app
    depends_on:
        db:
            condition: service_healthy
        redis:
            condition: service_healthy
    networks:

```

```

    - app_network
deploy:
  resources:
    limits:
      memory: 1G

celery_beat:
  build:
    context: .
    dockerfile: Dockerfile
  container_name: price_celery_beat
  restart: unless-stopped
  command: celery -A app.celery_app:celery_app beat --loglevel=info
  env_file: .env
  environment:
    - PYTHONUNBUFFERED=1
  volumes:
    - ./app
    - celery_beat_data:/app/celerybeat
  depends_on:
    db:
      condition: service_healthy
    redis:
      condition: service_healthy
  networks:
    - app_network

flower:
  build:
    context: .
    dockerfile: Dockerfile
  container_name: price_flower
  restart: unless-stopped
  command: >
    celery -A app.celery_app:celery_app flower
    --port=5555
    --basic-auth=${FLOWER_USER}:${FLOWER_PASSWORD}
  env_file: .env
  # NO ports exposed — accessed via Cloudflare Tunnel
  depends_on:
    - redis
  networks:
    - app_network

prometheus:
  image: prom/prometheus:latest
  container_name: price_prometheus
  restart: unless-stopped
  # NO ports exposed — accessed via Cloudflare Tunnel
  volumes:
    - ./prometheus/prometheus.yml:/etc/prometheus/prometheus.yml

```



```

    - promdata:/prometheus
depends_on:
    - api
networks:
    - app_network

grafana:
    image: grafana/grafana:latest
    container_name: price_grafana
    restart: unless-stopped
    # NO ports exposed — accessed via Cloudflare Tunnel
    env_file: .env
    environment:
        - GF_SECURITY_ADMIN_PASSWORD=${GRAFANA_ADMIN_PASSWORD}
        - GF_USERS_ALLOW_SIGN_UP=false
    volumes:
        - grafanadata:/var/lib/grafana
        - ./grafana/dashboards:/etc/grafana/provisioning/dashboards
        - ./grafana/datasources:/etc/grafana/provisioning/datasources
    depends_on:
        - prometheus
    networks:
        - app_network

volumes:
    pgdata:
    redisdata:
    promdata:
    grafanadata:
    model_cache:
    celery_beat_data:

networks:
    app_network:
        driver: bridge

```

Key changes from your original:

- Removed all ports: bindings — everything is internal-only, accessed via Cloudflare Tunnel
- Changed restart: always → restart: unless-stopped (won't auto-start after manual stop)
- Added memory limits per container to prevent one service from crashing the host
- Removed --reload from API (production mode)
- Increased Redis maxmemory to 512MB

6. Start the Stack

```
cd ~/price_comparision_engine
```

```
# Build and start everything
docker compose up --build -d

# Watch logs
docker compose logs -f

# Or watch specific service
docker compose logs -f api
docker compose logs -f celery_worker

# Check all services are healthy
docker compose ps

# Health check from inside the network
docker compose exec api curl -s http://localhost:8000/health
```

7. Verify Everything Works

Check	Command / URL	Expected Result
API Health	<code>curl https://api.pce-api.io/health</code>	<code>{"status": "healthy"}</code>
API Docs	<code>https://api.pce-api.io/docs</code>	Swagger UI loads
Grafana	<code>https://grafana.pce-api.io</code>	Login page, admin/ GRAFANA_ADMIN_PASSWORD
Flower	<code>https://flower.pce-api.io</code>	Celery task dashboard
Prometheus	<code>https://prometheus.pce-api.io</code>	Query interface
Database	<code>docker compose exec db psql -U price_engine -d price_comparison -c "\dt"</code>	Lists all 19 tables
Redis	<code>docker compose exec redis redis-cli -a your_password ping</code>	PONG
Celery Beat	<code>docker compose logs celery_beat tail -20</code>	Shows scheduled tasks

8. Automated Daily Backups

Create this script and run it via cron:

```
sudo mkdir -p /opt/pce-backup
sudo nano /opt/pce-backup/backup.sh
```

```
#!/bin/bash
# PCE Daily Backup Script

BACKUP_DIR="/opt/pce-backup/daily"
S3_BUCKET="s3://your-pce-backups" # Optional: configure AWS CLI
```

```

DATE=$(date +%Y%m%d_%H%M%S)
RETENTION_DAYS=7

mkdir -p $BACKUP_DIR

# PostgreSQL backup
docker compose -f /home/$USER/price_comparision_engine/docker-compose.yml exec -T db pg_dump -U
↪ price_engine -d price_comparison | gzip > $BACKUP_DIR/pce_db_$DATE.sql.gz

# Redis backup
docker compose -f /home/$USER/price_comparision_engine/docker-compose.yml exec -T redis redis-cli -a
↪ $(grep REDIS_URL /home/$USER/price_comparision_engine/.env | cut -d':' -f3 | cut -d'@' -f1) --rdb /
↪ data/dump.rdb
docker cp price_redis:/data/dump.rdb $BACKUP_DIR/pce_redis_$DATE.rdb

# .env backup
cp /home/$USER/price_comparision_engine/.env $BACKUP_DIR/pce_env_$DATE.env

# Upload to S3 (optional — install awscli and configure first)
# aws s3 sync $BACKUP_DIR $S3_BUCKET/daily/

# Clean old backups
find $BACKUP_DIR -name "pce_*" -mtime +$RETENTION_DAYS -delete

# Log
echo "[${DATE}] Backup completed: DB + Redis + ENV" >> /var/log/pce-backup.log

```

```

# Make executable
sudo chmod +x /opt/pce-backup/backup.sh

# Test it once
sudo /opt/pce-backup/backup.sh

# Add to crontab (runs daily at 3 AM)
(crontab -l 2>/dev/null; echo "0 3 * * * /opt/pce-backup/backup.sh") | crontab -

# Verify cron job
crontab -l

```

9. Uptime Monitoring (Free)

Sign up at uptimerobot.com (free tier):

Monitor	URL	Interval
API	https://api.pce-api.io/health	Every 5 minutes
Grafana	https://grafana.pce-api.io	Every 5 minutes
Flower	https://flower.pce-api.io	Every 5 minutes

Alert channels: Email + Telegram bot (free). You'll know within 5 minutes if your local server goes down.

10. Update Your Flutter App

In your Flutter app, update the base API URL:

```
// lib/services/api_service.dart or wherever you define the base URL

class ApiConfig {
  // Local development
  // static const String baseUrl = 'http://localhost:8000';

  // Production (your Cloudflare Tunnel)
  static const String baseUrl = 'https://api.pce-api.io';

  static const String apiVersion = '/api/v1';
}
```

Build and release the updated APK to your early users.

11. Resource Monitoring (Know Your Limits)

```
# Install htop for visual monitoring
sudo apt install -y htop

# Run this to see per-container resource usage
docker stats

# Key metrics to watch:
# - celery_worker memory: if it hits 3GB limit, reduce --concurrency from 2 to 1
# - db memory: if it grows beyond 4GB, you need more RAM or connection pooling tuning
# - api CPU: if consistently >80%, add --workers 4 (but needs more RAM)
```

When to know you're hitting local limits:

- Celery tasks queue depth >100 for >10 minutes → CPU bottleneck
 - PostgreSQL connections maxed out → need connection pooling (PgBouncer)
 - Docker containers getting OOMKilled → need more RAM or cloud migration
 - Scrape success rate drops below 80% → Playwright is overwhelmed, reduce catalog size
-

12. The Migration Playbook (Month 3)

When you're ready to move to cloud:

Pre-Migration (1 week before)

```
# 1. Take final local backup
sudo /opt/pce-backup/backup.sh

# 2. Note your latest schema version
docker compose exec db psql -U price_engine -d price_comparison -c "SELECT version FROM
↳ schema_migrations ORDER BY version DESC LIMIT 1;"
```

Cloud VPS Setup (Hetzner / DigitalOcean / AWS Lightsail)

```
# 1. Spin up a Ubuntu 24.04 VPS (16GB RAM, 4 vCPU minimum)
# 2. SSH into it
# 3. Repeat Steps 2, 3, 4, 5 from this guide
# 4. Copy your repo: scp -r price_comparison_engine/ root@your-vps-ip:/root/
# 5. Copy your .env: scp .env root@your-vps-ip:/root/price_comparison_engine/
# 6. Copy backups: scp /opt/pce-backup/daily/pce_db_LATEST.sql.gz root@your-vps-ip:/tmp/
```

Database Migration

```
# On the cloud VPS:
cd ~/price_comparison_engine
docker compose up -d db

# Wait 30 seconds for DB to initialize, then restore
zcat /tmp/pce_db_LATEST.sql.gz | docker compose exec -T db psql -U price_engine -d price_comparison

# Verify
docker compose exec db psql -U price_engine -d price_comparison -c "SELECT COUNT(*) FROM products;"
```

Cutover (Zero Downtime)

```
# 1. Start all services on cloud VPS
docker compose up -d

# 2. Update Cloudflare DNS
# In Cloudflare dashboard, change api.pce-api.io from Tunnel to A record pointing to VPS IP
# Or use cloudflared tunnel route dns with the new tunnel

# 3. DNS propagates in ~2 minutes
# 4. Test: curl https://api.pce-api.io/health
# 5. Shut down local server: docker compose down
```

Total migration time: 30 minutes.

13. Monthly Cost Comparison

Item	Local (Month 1-2)	Cloud VPS (Month 3+)	AWS Proper (Month 12+)
Server	0 (hardware you own)	4,500 (Hetzner CX51)	25,000+ (ECS + RDS + ElastiCache)
Domain	70/mo	70/mo	70/mo
Cloudflare	0	0	0 (Pro plan later)
S3	200	200	500
Back-ups			
Monitoring		0	0 (UptimeRobot free)
Electricity	500/mo	—	—
Internet	1,500/mo (existing)	—	—
TOTAL	1,770/mo	4,770/mo	25,570+/mo

Savings in first 2 months: 6,000 — not huge, but that's 2 months of runway preserved.

The real win: You prove traction before spending on cloud. Investors love capital efficiency.

14. Troubleshooting Common Issues

Issue: cloudflared fails to start

```
# Check logs
sudo journalctl -u cloudflared -n 50

# Common fix: wrong credentials path in config.yml
sudo nano /etc/cloudflared/config.yml
# Ensure credentials-file is absolute path
```

Issue: PostgreSQL fails to start

```
# Check if pgdata volume is corrupted
docker compose logs db

# Nuclear option: reset (DELETES ALL DATA)
docker compose down -v
docker compose up -d db
# Then restore from backup
```

Issue: Celery worker OOMKilled

```
# Reduce concurrency
docker compose exec celery_worker celery -A app.celery_app:celery_app inspect stats

# Edit docker-compose.yml: change --concurrency=2 to --concurrency=1
docker compose up -d --force-recreate celery_worker
```

Issue: API returns 502 via Cloudflare

```
# Cloudflare Tunnel can't reach the API
# Check if API container is healthy
docker compose ps api

# Check API logs
docker compose logs api | tail -50

# Verify internal network
docker compose exec cloudflared curl http://api:8000/health
# If this fails, the service name in config.yml is wrong
```

Issue: Playwright scrapes fail with timeout

```
# Check if target sites are blocking your IP
docker compose logs celery_worker | grep "timeout"

# Check your IP reputation
curl ipinfo.io

# If blocked, enable proxy in .env
# SCRAPER_PROXY_URL=http://your-proxy:8080
```

15. Security Checklist for Local Production

- .env file is not in git (git rm --cached .env if it is)
- .env permissions are 600 (chmod 600 .env)
- PostgreSQL is NOT exposed to internet (no ports in docker-compose)
- Redis is NOT exposed to internet (no ports in docker-compose)
- Redis has requirepass set
- JWT secret is 32+ random characters
- Admin password is bcrypt hashed (not plaintext)
- Flower has basic auth enabled
- Grafana has strong admin password
- Cloudflare Tunnel is the ONLY entry point
- Daily backups are running and tested
- UPS is connected and tested
- Server BIOS is set to auto-power-on after power loss

You're now ready to run PCE on local hardware for 2 months, then migrate to cloud with zero data loss.

Questions? Check `docker compose logs` first — it tells you everything.